

SUNLANDS ZHU GUAN TI HUI ZONG



主观题汇总

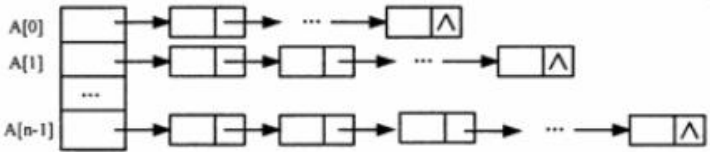
数据结构

目录

第二章 线性表.....	1
第三章 栈和队列.....	4
第四章 多维数组和广义表.....	5
第五章 树和二叉树.....	5
第六章 图.....	8
第七章 排序.....	10
第八章 查找.....	13

第二章 线性表

节	知识点名称	主观题
第二节 线性表的 顺序存储 和基本运 算的实现	线性表的顺 序存储 ★★★★ (算法阅读 题)	【真题 1810】顺序表类型定义如下： <pre># define ListSize 100 typedef struct { int data[ListSize]; int length; } SeqList;</pre> 阅读下列程序，并回答问题。 <pre>int partmin(SeqList *SL1, SeqList *SL2) { int minlength, minvalue, k = 0; minlength = SL2->length; minvalue = SL2->data[0]; while(k < minlength) { if (SL1->data[k] < SL2->data[k] && SL1->data[k] < minvalue) minvalue = SL1->data[k]; else if(SL2->data[k] < minvalue) minvalue = SL2->data[k]; k++; } return minvalue; } int f30(SeqList *SL1, SeqList *SL2) { if (SL1->length > SL2->length) return partmin(SL1, SL2); else return partmin(SL2, SL1); }</pre> 1. 若 SL1->data 中的数据为 (15, 14, 25, 8, -28, 37, 126, 56, 34)，SL2->data 中的数据为 (12, 7, -33, 15, 39, 24, 42, 13)，则调用函数 f30(&SL1, &SL2) 后的返回值是什么？ 答案：调用 f30 () 后的返回值是-33。 解析： 根据算法可知，找出两个线性表中前 8 个值中的最小值，应该是-33。 2. 该函数的功能是什么？ 答案： 该算法按两个线性表的最短长度 minlength 查找两个线性表中前 minlength 个元素的最小值。 解析： 根据题 1 的实例可得该函数功能。

第三节 线性表的 链式存储 结构	单链表（线性链表） ★★★★ (算法设计题)	【真题 1810】已知 n 个单链表的表头指针保存在数组 A 中，单链表中的结点类型及数组类型定义如下，存储形式如题图所示。 <pre> #define MAXSIZE 100 typedef int DataType; typedef struct node { DataType data; // data 是数据域 struct node *next; // 指向下一结点的指针 } Node; typedef Node * SeqList [MAXSIZE]; </pre>  试设计算法，在多个链表中查找值为 key 的数据元素，查找成功返回 1，查找失败返回 0。函数原型为 <code>int f34(Seqlist A, int n, int key)</code> 答案： <pre> int f34(SeqList A, int n, int key) { int i; Node * p; for(i = 0; i < n; i++) { p = A[i]; while(p!=NULL && p->data!=key) { p = p->next; } if(p!=NULL) return 1; } return 0; } </pre> 解析： 若 p 的数据域不为 key，则 p 指向下一个结点。直到查找值为 key 的数据元素，查找成功返回 1。 【真题 1610】已知单链表类型定义如下： <pre> typedef struct node { int data; struct node * next; } ListNode; typedef ListNode * List_ptr; </pre> 单链表 L 中结点数不少于 2。设计算法判断 L 中存储的全部 n 个数据是否是斐波那契序列的前 n 项。如果是，则函数返回 1，否
-------------------------------------	---------------------------------------	--

		则返回 0。函数原型如下： <pre>int IsF(List_ptr head); // 判定是否是斐波那契序列</pre> 注：斐波那契序列的定义为：$f_0=0$, $f_1=1$, $f_n=f_{n-1}+f_{n-2}$ ($n \geq 2$) 答案： <pre>int IsF(List_ptr head) { ListNode * pre1 = head->next, * pre2 = head->next->next, * current = pre2->next; if(pre1->data != 0 pre2->data != 1) return 0; while(current != NULL) { if(current->data == pre1->data + pre2->data) { pre1 = pre2; pre2 = current; current = current->next; } else break; } if(current != NULL) return 0; else return 1; }</pre> 解析：pre1、pre2、current 三个指针开始时分别指向第 1、2、3 三个结点，这三个指针扫描该单链表，扫描过程中，当第 1 个结点不等于 0 或第 2 个结点不等于 1 时，说明不满足斐波那契序列条件，返回 0；如果扫描完整个链表，pre1->data + pre2->data == current->data 成立，则满足斐波那契序列条件，返回 1。
	建立单链表 ★ (算法阅读题)	【真题 1510】请写出下列程序段的输出结果。 <pre>#define ListSize 100 typedef struct { int data[ListSize]; int length; } SeqList; void f30(SeqList * list) { int i, j, k; for(i = 0; i <= list->length - 2; i++) { j = i + 1; while(j <= list->length - 1) { if(list->data[i] == list->data[j]) { for(k = j; k < list->length - 1; k++) list->data[k] = list->data[k + 1]; list->length--; } else j++; } } }</pre>

		<pre> void main() { SeqList list = {0,3,7,3,3,3,4,0,3,7},10}; int i; f30(&list); printf("len = %d\n",list.length); for(i=0;i<list.length;i++) printf("%d,",list.data[i]); printf("\n"); } </pre> 运行结果: 答案: len=4 0, 3, 7, 4 解析: f30 函数的功能是将 list 的成员 data 数组中数值相同的元素只保留排在前面的, 后面的都删掉, 且没删掉 1 个元素, list.length 的值减 1。所以到最后 data 数组中的有效元素为 {0, 3, 7, 4}。
--	--	---

第三章 栈和队列

节	知识点名称	主观题
第一节 栈	栈的定义及其运算 ★★★★ (算法阅读题)	【真题 1610】阅读下列程序, 写出 f32 的输出结果。 <pre> void f32() { SeqStack *S; char x, y; InitStack(S); x = 'h'; y = 't'; Push(S, x); Push(S, 'c'); x = Pop(S); Push(S, x); Push(S, y); Push(S, 'a'); Push(S, x); while(!StackEmpty(S)) { y = Pop(S); printf("%c", y); } printf("%c\n", '!'); } </pre> 答案: 输出结果: catch! 解析:

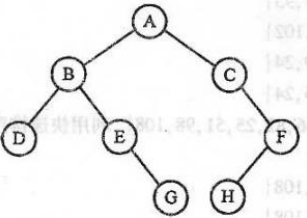
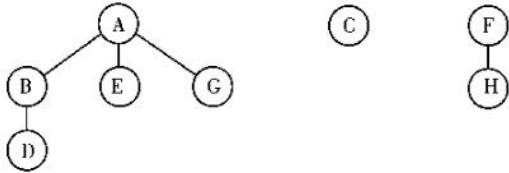
		题目很简单就是堆栈的先进后出。 进栈: Push (); 出栈: Pop ()
--	--	---

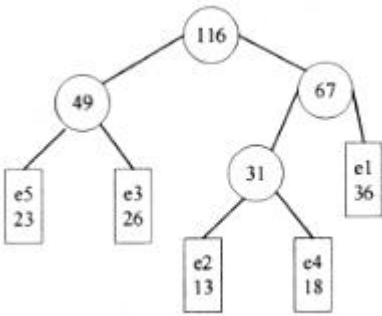
第四章 多维数组和广义表

节	知识点名称	主观题
第二节 矩阵的压缩存储	三角矩阵 ★ (解答题)	【真题 1510】已知 n 阶对称矩阵 A 的元素为 $a_{i,j}$ ($0 \leq i, j \leq n-1$), 采用“按行优先”将下三角部分的元素(含主对角线)保存在一维数组 sa 中, 且约定元素 $a_{0,0}$ 保存在 $sa[0]$ 中, 元素 $a_{i,j}$ ($0 \leq i, j \leq n-1$) 保存在 $sa[k]$ 中, 请给出由下标 i, j 计算下标 k 的计算公式。 答案: 设元素 $a_{i,j}$ ($0 \leq i, j \leq n-1$) 在数组 sa 中的存储位置为 k, 若 $i \geq j$, a 在下三角矩阵中, 因此有 $k = i(i+1)/2 + j$; 若 $i < j$, 则 a 在上三角矩阵中, 因此有 $k = j(j+1)/2 + i$。 $k = \begin{cases} \frac{i(i+1)}{2} + j & \text{当 } i \geq j \\ \frac{j(j+1)}{2} + i & \text{当 } i < j \end{cases}$ 即 解析: 当 $i \geq j$ 时, 第 0 行存储 1 个元素、第 1 行存储 2 个元素、第 2 行存储 3 个元素——第 $i-1$ 行存储 i 个元素, 所以元素 $a_{i,j}$ 前面已经存储了 $1+2+\dots+i$ 个元素, 元素 $a_{i,j}$ 是第 i 行的第 j 个元素, 所以元素 $a_{i,j}$ 保存在 $sa[k]$ 中下标 $k=1+2+\dots+i+j = i(i+1)/2+j$。当 $i < j$, j 存储在 a_{ji} 的位置, 此时 $k = j(j+1)/2+i$。

第五章 树和二叉树

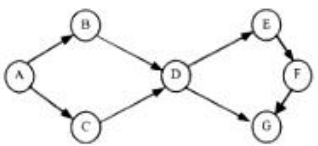
节	知识点名称	主观题
第二节 二叉树	链式存储结构 ★★★ (算法阅读题、算法设计题)	【真题 1710】已知二叉树的存储结构类型定义如下: <pre>typedef struct node { int data; struct node *lchild, *rchild; } BinNode; typedef BinNode *BinTree;</pre> 编写递归算法, 对于给定的一棵二叉树 T, 将其修改为镜像二叉树。例如, 题图所示的两棵二叉树互为镜像二叉树。 函数的原型为 <code>void f34(BinTree T);</code> 答案:

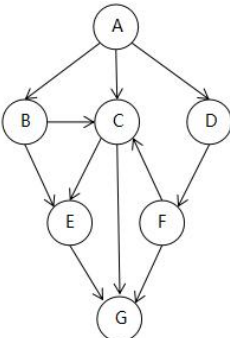
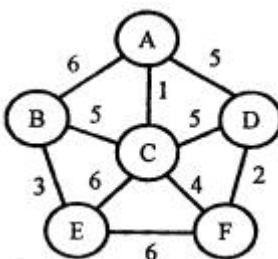
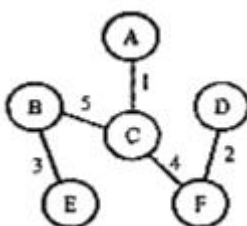
		<pre> void f34 (BinTree BT) { BinNode *s; if (BT) { s = BT->lchild; BT->lchild = BT->rchild; BT->rchild = s; f34 (BT->lchild); f34 (BT->rchild); } } </pre> 解析： 先设置一个中间变量 s，用来存放左子树的值，然后把右子树的值交换给左子树，再让右子树获得中间变量 s 值，这样做到左右子树结点值互换，左右子树分别递归，最后完成镜像。
第五节 树和森林	二叉树到 树、森林的 转换 ★★★★ (解答题)	【真题 1510】已知二叉树 T 如题图所示。  请问答下列问题： 1. 画出该二叉树对应的森林。 答案：  解析： 将一棵二叉树转换为树可以按下列步骤进行： ①若某结点是其双亲结点的左孩子，则将该结点的右孩子以及当且仅当连续地沿着此右孩子的右子树方向不断地搜索到的所有右孩子，都分别与该结点的双亲结点用虚线连接起来。 ②删去原二叉树中所有双亲结点与其右孩子的连线。 ③将图形规整化，使各结点按层次排列，并且将虚线改成实线。 2. 写出对森林进行前序遍历的遍历序列。 答案：ABDEGCFH 解析： 前序遍历森林的原则：若森林非空，则：①访问森林中第一棵树的根结点；②前序遍历第一棵树中根结点的各子树所构成的森林；③前序遍历除第一棵树外其它树。
第六节 哈夫曼树	哈夫曼算法 ★★★★	【真题 1810】设电文字符集是 $\{e_1, e_2, e_3, e_4, e_5\}$，各字符出现的次数分别为 $\{36, 13, 26, 18, 23\}$。现要为该字符集设计哈

及其应用	(解答题、算法阅读题)	夫曼编码。请回答下列问题。 1. 给出构造的哈夫曼树。 答案：  解析： 哈夫曼树的构造方法： 1) 根据与 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 对应的 n 个结点构成 n 棵二叉树的森林 $F = \{T_1, T_2, \dots, T_n\}$ 其中每棵二叉树 T_i 都只有一个权值为 w_i 的根结点，其左、右树均为空。 2) 在森林 F 中选出两棵根结点的权值最小的树作为一棵新树的左、右子树，且置新树的附加根结点的权值为其左、右子树上根结点的权值之和。 3) 从 F 中删除这两棵树，同时把新树加入到 F 中； 4) 重复步骤 (2) 和 (3)，直到 F 中只有一棵树为止，此树便是哈夫曼树。 注意：本题答案不唯一，考生给出的答案正确同样给分。 2. 给出各字符的哈夫曼编码。 答案： <table><tr><td>字符</td><td>e_1</td><td>e_2</td><td>e_3</td><td>e_4</td><td>e_5</td></tr><tr><td>编码</td><td>11</td><td>100</td><td>01</td><td>101</td><td>00</td></tr></table> 解析： 树中从根到每个叶子都有一条路径，对路径上的各分支约定指向左子树的分支表示“0”码，指向右子树的分支表示“1”码，取每条路径上的“0”或“1”的序列作为各个叶子结点对应的字符编码，这就是哈夫曼编码。 注意：本题答案不唯一，考生给出的答案正确同样给分。 3. 计算电文编码总长。 答案： 电文编码总长=$36 \times 2 + 13 \times 3 + 26 \times 2 + 18 \times 3 + 23 \times 2 = 263$ 解析： 假设每种字符在电文中出现的次数为 W_i，编码长度为 L_i，电文	字符	e_1	e_2	e_3	e_4	e_5	编码	11	100	01	101	00
字符	e_1	e_2	e_3	e_4	e_5									
编码	11	100	01	101	00									

		中有 n 种字符，则电文编码总长为 $\sum W_i L_i$。若将此对应到二叉树上，W_i 为叶结点的权，L_i 为根结点到叶结点的路径长度。那么，$\sum W_i L_i$ 恰好为二叉树上带权路径长度。
--	--	---

第六章 图

节	知识点名称	主观题
第二节 图的存储结构	邻接矩阵表示法 ★★★ (解答题)	【真题 1810】已知图 G 采用邻接矩阵存储，邻接矩阵如题图所示。 $ \begin{array}{c} A \ B \ C \ D \ E \ F \ G \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \\ G \end{matrix} \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array} $ 1. 根据邻接矩阵画出图 G。 答案：  解析： 邻接矩阵：用矩阵来描述图中顶点之间的关联关系。其中： $ A[i][j] = \begin{cases} 1 & \text{若 } (v_i, v_j) \text{ 或 } \langle v_i, v_j \rangle \text{ 是 } E \text{ 中的边} \\ 0 & \text{若 } (v_i, v_j) \text{ 或 } \langle v_i, v_j \rangle \text{ 不是 } E \text{ 中的边} \end{cases} $
第三节 图的遍历	深度优先搜索遍历 ★★ (解答题)	假定以图中某个顶点 v_0 为出发点，首先访问出发点 v_i，然后任选一个 v_i 的未访问过的邻接点 v_j，以 v_j 为新的出发点继续进行深度优先搜索，依此类推，直至图中所有顶点都被访问过。
	广度优先搜索遍历 ★★ (解答题)	【真题 1710】已知图 G 采用邻接矩阵存储，邻接矩阵如题图所示。 $ \begin{array}{c} A \ B \ C \ D \ E \ F \ G \\ \begin{matrix} A \\ B \\ C \\ D \\ E \\ F \\ G \end{matrix} \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array} $ 1. 写出从顶点 A 开始图 G 的 3 个不同的深度优先搜索遍历序列。 答案：ABCEGDF、ACEGBDF、ADFGBCE 解析：根据邻接矩阵可以画出这个有向图（因为不是对称矩阵）：

		 连通图深度优先搜索的基本思想：假定以图中某个顶点 v_0 为出发点，首先访问出发点 v_i，然后任选一个 v_i 的未访问过的邻接点 v_j，以 v_j 为新的出发点继续进行深度优先搜索，以此类推，直至图中所有顶点都被访问过。故可得出答案，注意答案不唯一。 2. 写出从顶点 A 开始图 G 的 2 个不同的广度优先搜索遍历序列。 答案：ABCDEF G、ADCBFEG 解析：广度优先搜索按照第 1 层，第 2 层，第 3 层…的顺序访问，在每个层次中先被访问的顶点，其邻接点也先被访问。 故由上图可得出答案，注意答案不唯一。
第四节 图的生成 树和最小 生成树	最小生成树 ★★ (解答题)	构造最小生成树的常用算法：普里姆 (Prim) 算法和克鲁斯卡尔 (Kruskal) 算法。 【真题 1704】对题图所示的带权无向图 G，试回答以下问题。  1. 画出 G 的最小生成树。 答案：最小生成树：  解析： 对于连通的带权图（网）G，其生成树也是带权的。我们把生成树各边的权值总和称为该树的权，把权值最小的生成树称为图的最小生成树。 2. 若用克鲁斯卡尔 (Kruskal) 算法求最小生成树，请按被选中的次序写出最小生成树上各条边的顶点和权值。 答案：<A, C>1, <D, F>2, <B, E>3, <C, F>4, <B, C>5 解析：

		该算法的基本思想是：将图 G 中的边按权值从小到大的顺序依次选取 E 中的边 (U, V)，若选取的边使生成树 T 不形成回路，则把它并入 TE 中，保留作为 T 的一条边；若选取的边使生成树 T 形成回路，则将其舍弃，如此进行下去直到 TE 中包含条边为止，此时的 T 即为最小生成树。
--	--	--

第七章 排序

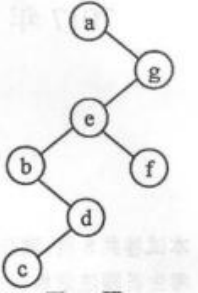
节	知识点名称	主观题
第二节 插入排序	直接插入排序 ★★ (算法阅读题)	【真题 1810】待排序的记录的数据类型定义如下： <pre> #define MAXSIZE 100 typedef int KeyType; typedef struct { KeyType key; } RecType; typedef RecType SeqList [MAXSIZE]; </pre> 下列函数实现顺序表的直接插入排序，请在空白处填上适当内容使算法完整。 <pre> void f32(SeqList R, int n) { int i, j; RecType temp; for (i=1; i<= (1) ; i++) { temp = R[i]; j = i; while (j > 0 && temp.key < R[j-1].key) { R[j] = R[j-1]; (2) ; } (3) ; } } </pre> 答案：(1) n-1; (2) j--; (3) R [j]= temp ; 解析： (1) 直接插入排序需要经过 n-1 次插入，故从 1 开始，到 n-1，共插入 n-1 次。 (2) 直接插入排序基本操作是将一条记录插入到已排好的有序表中，从而得到一个新的、记录数量增 1 的有序表。 故 j 递减，比较下一个元素。 (3) 如果将一个待排序的表看成两部分，第一部分就是第一个元素。可以把它看成是一个有序表，除了第一个元素之外的其他元素看成是第二部分。该排序方法就是将第二部分中的每个元素依次插入到前面的那个有序表中。
	希尔排序 ★★ (解答题)	【真题 1710】有以下数据序列(19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79, 12),使用希尔排序方法将其排成升序序列。请回答下列问题。 1. 写出增量为 4 时对上述数据序列进行一趟希尔排序的结果。 答案：12, 11, 10, 01, 19, 14, 23, 27, 55, 20, 84, 79, 68 解析：

		希尔排序又称“缩小增量排序”，其基本思想是：先取定一个小于 n 的整数 d_1 作为第一个增量，把数组 R 中的全部元素分成 d_1 个组，所有下标距离为 d_1 的倍数的元素放在同一组中，即 $R[1]$, $R[1+d_1]$, $R[1+2d_1]$, ... 为第一组, $R[2]$, $R[2+d_1]$, $R[2+2d_1]$, ... 为第二组, ..., 接着在各组内进行直接插入排序；然后再取 d_2 ($d_2 < d_1$) 为第二个增量，重复上述分组和排序，直到所取的增量 $dt=1$ ($dt < dt-1 < \dots < d_2 < d_1$) 把所有的元素放在同一组中进行直接插入排序为止。 本题中，$d_1=4$, $R[1]$, $R[5]$, $R[9]$, $R[13]$ 为一组；$R[2]$, $R[6]$, $R[10]$ 为一组；$R[3]$, $R[7]$, $R[11]$ 为一组；$R[4]$, $R[8]$, $R[12]$ 为一组。在各组内进行直接插入排序得到答案。 2. 给出一个可行的希尔排序增量序列。 答案：4, 2, 1 解析：希尔排序的思想，需要取一个小于 n 的整数做为第一个增量，因为增量的不确定性，所以本题的答案也不唯一，如果增量序列是递减，且最后一个增量为 1 也是正确的。
第三节 交换排序	冒泡排序 ★★★ (算法阅读题、算法设计题)	【真题 1710】待排序记录的数据类型定义如下： <pre> #define MAXSIZE 100 typedef int KeyType; typedef struct { KeyType key; } RecType; typedef RecType SeqList [MAXSIZE]; </pre> 下列算法实现自底向上、自顶向下交替进行的双向扫描冒泡排序，请在空白处填上适当内容使算法完整。 <pre> void f32 (SeqList R, int n) { int i=0, j; RecType t; int NoSwap=1; while(NoSwap){ NoSwap=0; for(j = n-i-1; ____ (1) ____) if(R[j].key < R[j-1].key){ t=R[j]; R[j]=R[j-1]; R[j-1]=t; NoSwap=1; } for(____ (2) ____; j++) if(R[j].key > R[j+1].key){ t=R[j]; R[j]=R[j+1]; R[j+1]=t; NoSwap=1; } ____ (3) ____; } } </pre> 答案：(1) $j \geq i+1; j--$; (2) $j=j+1; j < n-i-1$ (3) $i++$

		解析： (1) 第一个空是自底向上扫描 (2) 第二个空是自顶向下扫描。 (3) $i=i+1$ 也可。实现依次类推，一直到所有的记录都有序为止。 【真题 1510】已知顺序表 SeqList 定义如下： <pre>typedef struct{ KeyType key; InfoType otherinfo; }RecType; typedef RecType SeqList[MAXSIZE+1];</pre> 编写函数，用冒泡排序法将 n 个元素的待排序列 R 按关键字降序排序。函数原型为： <pre>int f34(SeqList R, int n)。</pre> 答案： <pre>{int i, j, flag; for(i=1; i<n; i++) //最多做 n-1 趟排序 { flag=0; //flag 表示每一趟是否有交换，先置 0 for(j=n; j>=i+1; j--) //进行第 i 趟排序 if(R[j].key>R[j-1].key) { R[0]=R[j-1]; //R[0]作为交换时的暂存单元 R[j-1]=R[j]; R[j]=R[0]; //相邻两个元素完成交换 flag=1; //有交换，flag 置 1 } if(flag==0) return; } }</pre> 解析： 冒泡排序过程具体描述为：首先将 $R[n].key$ 和 $R[n-1].key$ 进行比较，若 $R[n].key < R[n-1].key$，则交换 $R[n]$ 和 $R[n-1]$，使轻者上浮，重者下沉；接着比较 $R[n-1].key$ 和 $R[n-2].key$，同样使轻者上浮，重者下沉，依此类推，直到比较 $R[2].key$ 和 $R[1].key$，若反序则交换，第一趟排序结束，此时，记录 $R[1]$ 的关键字最小。然后再对 $R[n] \sim R[2]$ 的记录进行第二趟排序，使次小关键字的元素被上浮到 $R[2]$ 中，重复进行 $n-1$ 趟后，整个冒泡排序结束。注意本题中要求降序排序，故要注意这条语句 <code>if (R [j].key)> R [j-1].key)</code>。
第四节 选择排序	直接选择排序 ★★ (解答题)	【真题 1610】对于给定的一组关键字序列 {26, 18, 60, 65, 45, 13, 32}，写出使用直接选择排序方法将其排成升序序列的过程。 答案： 原始序列：26, 18, 60, 65, 45, 13, 32 第一趟排序后：13, 18, 60, 65, 45, 26, 32 第二趟排序后：13, 18, 60, 65, 45, 26, 32 第三趟排序后：13, 18, 26, 65, 45, 60, 32 第四趟排序后：13, 18, 26, 32, 45, 60, 65

		第五趟排序后：13, 18, 26, 32, 45, 60, 65 第六趟排序后：13, 18, 26, 32, 45, 60, 65 解析： 直接选择排序是一种简单的排序方法。它的基本思想是：每次从待排序的无序区中选择出关键字值最小的记录，将该记录与该区中的第一个记录交换位置。初始时，$R[1 \cdots n]$为无序区，有序区为空。第一趟排序是在无序区$R[1 \cdots n]$中选出最小的记录，将它与$R[1]$交换，$R[1]$为有序区；第二趟排序是在无序区$R[2 \cdots n]$中选出最小的记录与$R[2]$交换，此时$R[1 \cdots 2]$为有序区；依此类推，做$n-1$趟排序后，区间$R[1 \cdots n]$中记录递增有序。
--	--	--

第八章 查找

节	知识点名称	主观题
第二节 顺序表的查找	顺序查找 ★★ (算法阅读题)	【真题 1610】阅读程序，回答下列问题。 <pre> int f33 (NodeType R[], KeyType k, int n) { int i = n-1, count = 1; R[0].key = k; while (R[i].key != k) { i--; count++; } if (i == 0) return -1; else return count; } </pre> 1. 变量 count 的含义是什么？ 答案：count 记录的是，从数组最后开始向前顺序查找时，找到目标所需的比较次数。 解析：这是一个顺序查找的程序段，如果查找成功，返回查找该关键字的过程中比较的个数或者是倒数第 count 个关键字；如果查找不成功，返回-1。 2. f33 的功能是什么？ 答案：在数组中，从后向前顺序查找，返回-1 表示查找不成功，返回正整数表示查找成功，且这个值表示进行查找的比较次数。 解析：在$R[]$中查找关键字k，如果查找成功，返回查找关键字k的过程中比较的次数或者说是关键字k是倒数第 count 个关键字；如果查找不成功，返回-1。
第三节 树表的查找	二叉排序树的插入和生成 ★★★★ (解答题、算法阅读题)	【真题 1710】设有二叉排序树如题图所示。请回答下列问题。 

		1. 假定二叉排序树初始为空，写出一个数据输入序列，按序插入时能得到题图所示的二叉排序树。 答案：a g e b f d c 解析：从上到下遍历，有左右子树，则会有不同的序列。本题遍历可以得到 agebfdc。以 e 为根的树有左右子树，其左右子树之间序列位置可以调换。 2. 能得到题图所示的二叉排序树的不同的输入数据序列有几个？ 答案：4 个 解析：遍历能得到题图所示的二叉排序树的不同的输入数据序列 agebfdc；agebdfc；agebdcf；agefbdc；共四个。
第三节 树表的查找	B 树上的插入和删除 ★★ (解答题)	【真题 1510】题图所示为一棵含 2 个关键字的 3 阶 B 树 T。现将关键字序列 {40, 60, 70, 20, 10} 依次插入到 T 中，画出每插入一个关键字后得到的树型。 30 50 答案： 40 30 50 插入 40 40 60 20 30 50 70 插入 20 40 20 60 10 30 50 70 插入 10 40 30 50 60 插入 60 40 60 30 50 70 插入 70 解析： 在 B 树中插入一个结点的过程：先在最低层的某个非终端结点中添加一个关键字。若该结点中关键字的个数不超过 $m-1$，则插入完成，否则要产生结点“分裂”。“分裂”结点时把结点分成两个，将中间的一个关键字拿出来插入到该结点的双亲结点上，如果双亲结点中已有 $m-1$ 个关键字，则插入后将引起双亲结点的分裂，这一过程可能波及 B 树的根结点，引起根结点的分裂，从而使 B 树长高一层。
第四节 散列表查找	开放定址法 ★★★ (解答题)	【真题 1704】已知散列表的长度为 11，散列函数为 $H(\text{key})=\text{key}\%11$，散列表的当前状态如下： 下标 0 1 2 3 4 5 6 7 8 9 10 关键字 60 17 29 现要插入关键字 38，回答下列问题。 1. 若用线性探查法解决冲突，则 38 所在位置的下标是什么？ 答案：插入位置的下标是 8。 解析：

		应用线性探测法解决冲突时，是在散列函数 H 得出的地址基础上 +1，若仍然冲突，继续找+2 的地址，直到不冲突为止。 本题中，散列函数 $H(key)=key\%11$，故 $38\%11=5$，即 38 应放到地址为 5 位置，但地址 5,6,7 都有元素，故元素 38 放到地址 8 的位置。 2. 若用二次探查法解决冲突，则 38 所在位置的下标是什么？ 答案：插入位置的下标是 4。 解析： 二次探查法的探查序列是：$hi = (H(key) \pm i^2) \% m (0 \leq i \leq m-1)$ 即探查序列为 $d = H(key), d+1^2, d-1^2, d+2^2, d-2^2, \dots$ 等。也就是说，探查从地址 d 开始，先探查 $T[d]$，然后再依次探查 $T[d+1^2], T[d-1^2], T[d+2^2], T[d-2^2], \dots$。 $d=H(key)=key\%11=38\%11=5$，故先探查地址 5，冲突；再探查地址 $d+1^2=6$，冲突；再探查地址 $d-1^2=4$。故元素 38 放到地址 4 的位置。 3. 以上两种方法中，各需要多少次探查次数？ 答案：探查次数分别是 4 和 3。 解析： 应用线性探测法解决冲突时，探查了地址 5,6,7,8，共 4 次。 用二次探查法解决冲突时，探查了地址 5,6,4，共 3 次。
--	--	---